RANDOM FOREST

AIM

To implement the Random Forest algorithm for classification tasks and evaluate its performance using appropriate classification metrics.

THEORY

A Random Forest is a supervised ensemble learning algorithm used for both classification and regression tasks. It builds multiple decision trees during training and combines their outputs to produce a more accurate and stable prediction. Instead of relying on a single tree, Random Forest aggregates the results of many trees to reduce overfitting and improve generalization.

The main components of a Random Forest are:

1) Decision Trees: Individual models trained on different subsets of data.

2) Bootstrap Samples: Random subsets of the original dataset selected with re-placement.

3) Feature Randomness: A random subset of features is considered at each split to ensure diversity among trees.

Each tree in the forest makes a prediction, and the final output is determined by majority voting (for classification) or averaging (for regression).

A Random Forest works by constructing multiple decision trees using different bootstrap samples of the training data. For each tree, at every node, only a random subset of features is considered when selecting the best split. This randomness ensures that the trees are diverse and not highly correlated. The splitting criteria commonly used include Gini Index and Entropy. The Gini Index measures the impurity of a node, representing the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution in the node. The Gini Index is calculated as:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{n} p_i^2$$

Lower Gini values indicate purer nodes, with a value of zero when all samples belong to one class. However, it is slightly biased toward dominant classes.

Entropy, on the other hand, measures the amount of uncertainty or disorder in a node and is used to calculate Information Gain and is computed as :

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

Entropy is zero for a perfectly pure node, with higher values indicating greater disorder. It is sensitive to small changes in class distribution and often produces balanced and informative splits. Once all trees are trained, predictions are made by combining the outputs of individual

trees. In classification, the class predicted by the majority of trees is selected, while in regression, the average of all tree predictions is taken as the final output. By combining multiple trees, Random Forest reduces variance, minimizes overfitting, and provides higher accuracy compared to a single decision tree..

Merits of Random Forest

1. **High Accuracy & Robustness** – By combining multiple decision trees, Random Forest reduces overfitting and usually gives better accuracy than a single decision tree.

2. **Handles Large & Complex Data** – It works well with large datasets and can handle both numerical and categorical features effectively.

3. **Feature Importance Estimation** – Random Forest can measure the importance of each feature, helping in feature selection and understanding the model.

Demerits of Random Forest

1. **Computationally Expensive** – Training many trees requires more time and memory compared to simpler models.

2. **Less Interpretability** – The ensemble nature makes it difficult to interpret compared to a single decision tree.

3. **Not Ideal for Real-Time Prediction** – Due to multiple trees, prediction can be slower, making it less suitable for time-critical applications.

CODE FOR BINARY CLASSIFICATION

INPUT: Import Libraries

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import matplotlib.pyplot as plt

import seaborn as sns

print("Libraries imported successfully\n")
```

OUTPUT:

Libraries imported successfully

INPUT: Load Dataset

```
df = pd.read_csv("bank_marketing.csv", sep=';')

print("Dataset loaded successfully")
```

OUTPUT:

Dataset loaded successfully

INPUT: Check The Data Shape

```
print("Dataset shape:", df.shape)
```

OUTPUT:

Dataset shape: (41188, 21)

INPUT: Display First 5 rows

```
print("\nFirst 5 rows:")

print(df.head(), "\n")
```

OUTPUT:

```
First 5 rows:

   age      job  marital    education  default housing loan    contact  \

0   56  housemaid  married     basic.4y      no      no   no  telephone

1   57   services  married  high.school  unknown      no   no  telephone

2   37   services  married  high.school      no     yes   no  telephone

3   40    admin.  married     basic.6y      no      no   no  telephone

4   56   services  married  high.school      no      no  yes  telephone


  month day_of_week  ...  campaign  pdays  previous     poutcome emp.var.rate  \

0   may       mon  ...         1    999         0  nonexistent          1.1

1   may       mon  ...         1    999         0  nonexistent          1.1
```

```
2   may       mon  ...       1   999       0  nonexistent       1.1
3   may       mon  ...       1   999       0  nonexistent       1.1
4   may       mon  ...       1   999       0  nonexistent       1.1

   cons.price.idx  cons.conf.idx  euribor3m  nr.employed   y
0          93.994          -36.4      4.857       5191.0  no
1          93.994          -36.4      4.857       5191.0  no
2          93.994          -36.4      4.857       5191.0  no
3          93.994          -36.4      4.857       5191.0  no
4          93.994          -36.4      4.857       5191.0  no
```

INPUT: Display Last 5 rows

```
print("\nLast 5 rows:")
print(df.tail(), "\n")
```

OUTPUT:

```
Last 5 rows:
      age        job  marital          education default housing loan  \
41183   73    retired  married  professional.course     no     yes   no
41184   46  blue-collar  married  professional.course     no      no   no
41185   56    retired  married   university.degree     no     yes   no
41186   44  technician  married  professional.course     no      no   no
41187   74    retired  married  professional.course     no     yes   no

       contact month day_of_week  ...  campaign  pdays  previous  \
41183  cellular   nov       fri  ...       1   999       0
41184  cellular   nov       fri  ...       1   999       0
```

41185 cellular nov fri ... 2 999 0

41186 cellular nov fri ... 1 999 0

41187 cellular nov fri ... 3 999 1

poutcome emp.var.rate cons.price.idx cons.conf.idx

41183 nonexistent -1.1 94.767 -50.8 1.028

41184 nonexistent -1.1 94.767 -50.8 1.028

41185 nonexistent -1.1 94.767 -50.8 1.028

41186 nonexistent -1.1 94.767 -50.8 1.028

41187 failure -1.1 94.767 -50.8 1.028

euribor3m nr.employed y

41183 4963.6 yes

41184 4963.6 no

41185 4963.6 no

41186 4963.6 yes

41187 4963.6 no

INPUT: Dataset Information

print("Step 3: Dataset information")

print(df.info(), "\n")

OUTPUT:

Data columns (total 21 columns):

Column Non-Null Count Dtype

0 age 41188 non-null int64

1 job 41188 non-null object

2 marital 41188 non-null object

```
 3   education      41188 non-null  object
 4   default        41188 non-null  object
 5   housing        41188 non-null  object
 6   loan           41188 non-null  object
 7   contact        41188 non-null  object
 8   month          41188 non-null  object
 9   day_of_week    41188 non-null  object
 10  duration       41188 non-null  int64
 11  campaign       41188 non-null  int64
 12  pdays          41188 non-null  int64
 13  previous       41188 non-null  int64
 14  poutcome       41188 non-null  object
 15  emp.var.rate   41188 non-null  float64
 16  cons.price.idx 41188 non-null  float64
 17  cons.conf.idx  41188 non-null  float64
 18  euribor3m      41188 non-null  float64
 19  nr.employed    41188 non-null  float64
 20  y              41188 non-null  object
```

INPUT: Categorial Encoding

```
df_encoded = df.copy()
le = LabelEncoder()

for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = le.fit_transform(df_encoded[col])
        print(f"Encoded column: {col}")
```

```
print("\nCategorical encoding completed\n")
```

OUTPUT:

```
Encoded column: job

Encoded column: marital

Encoded column: education

Encoded column: default

Encoded column: housing

Encoded column: loan

Encoded column: contact

Encoded column: month

Encoded column: day_of_week

Encoded column: poutcome

Encoded column: y

Categorical encoding completed
```

INPUT: Check the Shape of Features and Target

```
X = df_encoded.drop('y', axis=1)

y = df_encoded['y']


print("Features shape:", X.shape)

print("Target shape:", y.shape, "\n")
```

OUTPUT:

```
Features shape: (41188, 20)

Target shape: (41188,)
```

INPUT: Split the Data

```
X_train, X_test, y_train, y_test = train_test_split(

    X, y,
```

```
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape, "\n")
```

OUTPUT:

```
Training set shape: (32950, 20)
Testing set shape: (8238, 20)
```

<u>GINI</u>

INPUT: Initialization Of Random Forest Model

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
print("Random Forest model initialized\n")
```

OUTPUT:

```
Random Forest model initialized
```

INPUT: Predictions

```
y_pred = rf_model.predict(X_test)
print("Predictions completed\n")
```

OUTPUT:

Predictions completed

INPUT: Check The Accuracy

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy of Random Forest model:", accuracy, "\n")
```

OUTPUT:

```
Accuracy of Random Forest model: 0.9225540179655256
```

INPUT: Evaluation Of Model Performance

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n")

print(cm)
```

OUTPUT:

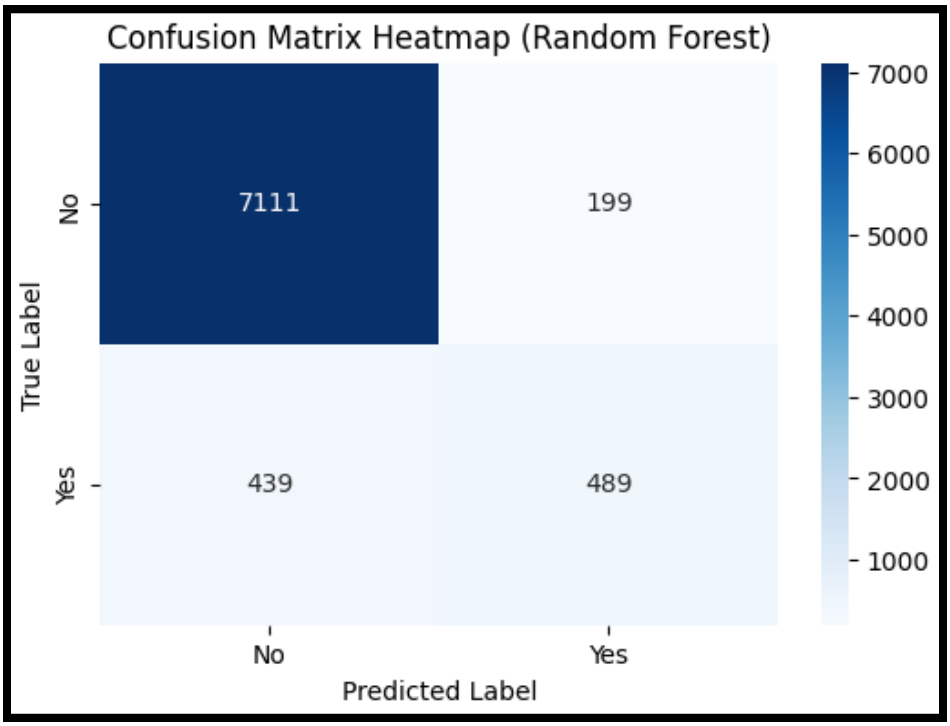
```
Confusion Matrix:

[[7111  199]

 [ 439  489]]
```

INPUT: Confusion Matrix Heatmap

```
print("\nStep 11(a): Confusion Matrix Heatmap")


import matplotlib.pyplot as plt

import seaborn as sns


plt.figure(figsize=(6, 4))

sns.heatmap(

    cm,

    annot=True,

    fmt='d',

    cmap='Blues',
```

```
    xticklabels=['No', 'Yes'],
    yticklabels=['No', 'Yes']
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap (Random Forest)")
plt.show()
```

OUTPUT:



<u>ENTROPY</u>

INPUT: Initialization Of Random Forest Model

```
rf_model = RandomForestClassifier(
 criterion='entropy',
   n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=2,
```

```
    random_state=42,
    n_jobs=-1
)
print("Random Forest model initialized\n")
```

OUTPUT:

```
Random Forest model initialized
```

INPUT: Check The Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Overall Accuracy: {accuracy:.4f}")
```

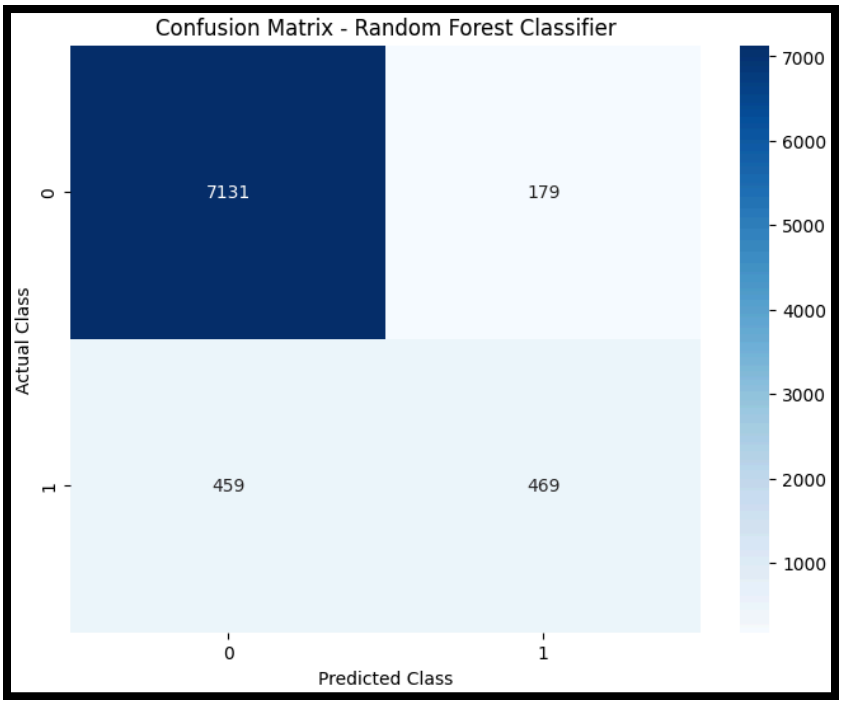
OUTPUT:

```
Overall Accuracy: 0.9226
```

INPUT: Confusion Matrix Heatmap

```
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=class_labels,
    yticklabels=class_labels
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Random Forest Classifier")
plt.show()
```

OUTPUT:



<u>CODE FOR MULTI CLASSIFICATION</u>

INPUT: Import Libraries

```
# Core Libraries

import pandas as pd

import numpy as np

# Machine Learning

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Visualization

import matplotlib.pyplot as plt

import seaborn as sns

print("Libraries imported successfully\n")
```

OUTPUT:
Libraries imported successfully

INPUT: Loading Dataset

```
df = pd.read_csv(
```

```
    "seeds_dataset.txt",
    sep=r"\s+",
    header=None,
    names=column_names
)
print("Dataset loaded successfully")
```

OUTPUT:

Dataset loaded successfully.

INPUT: Check the Data Shape

```
print("Dataset Shape:", df.shape)
```

OUTPUT:

Dataset Shape: (210, 8)

INPUT: Display first 5 rows

```
print(df.head())
```

OUTPUT:

```
First 5 Records:
    Area  Perimeter  Compactness  Length_of_kernel  Width_of_kernel  \
0  15.26      14.84       0.8710             5.763            3.312
1  14.88      14.57       0.8811             5.554            3.333
2  14.29      14.09       0.9050             5.291            3.337
3  13.84      13.94       0.8955             5.324            3.379
4  16.14      14.99       0.9034             5.658            3.562

   Asymmetry_coefficient  Length_of_kernel_groove  Class
0                  2.221                    5.220      1
1                  1.018                    4.956      1
```

2	2.699	4.825	1
3	2.259	4.805	1
4	1.355	5.175	1

INPUT: Data Split

```
# Separate features and target
X = df.drop("Class", axis=1)
y = df["Class"]
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
print("Data Splitting Complete” )
```

OUTPUT:

Data Splitting Complete

<u>GINI</u>

INPUT: MODEL TRAINING

```
# Initialize Random Forest with GINI criterion
rf = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    random_state=42,
    n_jobs=-1
```

```
)

# Train the model

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

print("Data Training Complete” )
```

OUTPUT:

Data Training Complete

INPUT: Model Evaluation

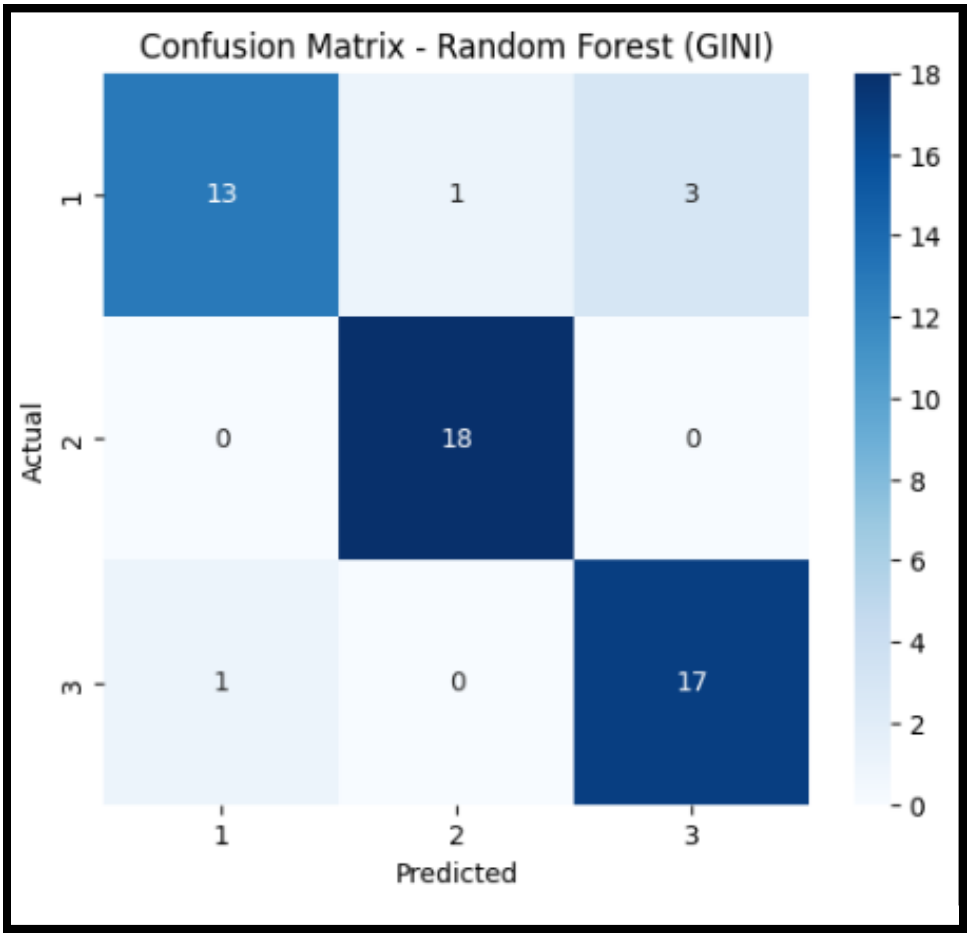
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nClassification Report:\n")

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))

sns.heatmap(

    cm,

    annot=True,

    fmt="d",

    cmap="Blues",

    xticklabels=np.unique(y),

    yticklabels=np.unique(y)

)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix - Random Forest (GINI)")

plt.show()
```

OUTPUT:

Accuracy: 0.9056603773584906

Classification Report

Class	Precision	Recall	F1-Score	Support
1	0.93	0.76	0.84	17
2	0.95	1.00	0.97	18
3	0.85	0.94	0.89	18
Accuracy			**0.91**	**53**
Macro Avg	**0.91**	**0.90**	**0.90**	**53**
Weighted Avg	**0.91**	**0.91**	**0.90**	**53**



<u>HYPERPERAMETER(GINI):</u>

INPUT: n_estimator

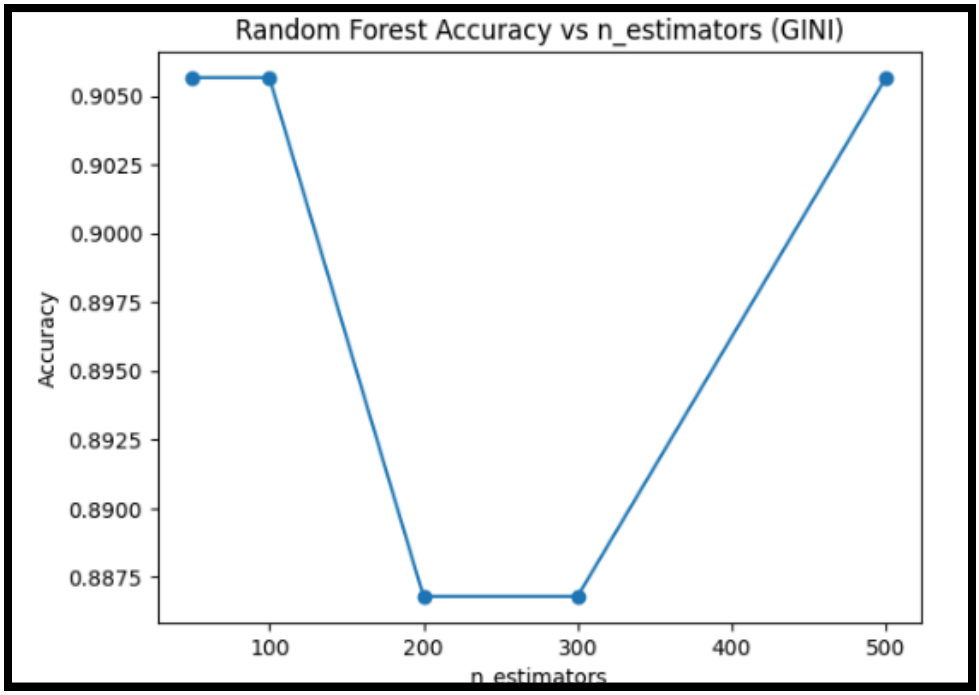
```
# n_estimators values
n_values = [50, 100,200,300,500]
accuracies = []
# Train & evaluate
for n in n_values:
    model = RandomForestClassifier(
        n_estimators=n,
        criterion="gini",
        random_state=42
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f"GINI | n_estimators={n} | Accuracy={acc:.4f}")
# Plot graph
plt.figure()
plt.plot(n_values, accuracies, marker='o')
plt.xlabel("n_estimators")
plt.ylabel("Accuracy")
plt.title("Random Forest Accuracy vs n_estimators (GINI)")
plt.show()
```

OUTPUT:

```
GINI | n_estimators=50 | Accuracy=0.9057
GINI | n_estimators=100 | Accuracy=0.9057
GINI | n_estimators=200 | Accuracy=0.8868
```

GINI | n_estimators=300 | Accuracy=0.8868

GINI | n_estimators=500 | Accuracy=0.9057



<u>ENTROPY</u>

INPUT: Model Training

```
# Initialize Random Forest with ENTROPY criterion

rf_entropy = RandomForestClassifier(

    n_estimators=200,

    criterion="entropy",

    random_state=42,

    n_jobs=-1

)

# Train the model

rf_entropy.fit(X_train, y_train)

y_pred_entropy = rf_entropy.predict(X_test)

print("Data Training Complete” )
```

OUTPUT:

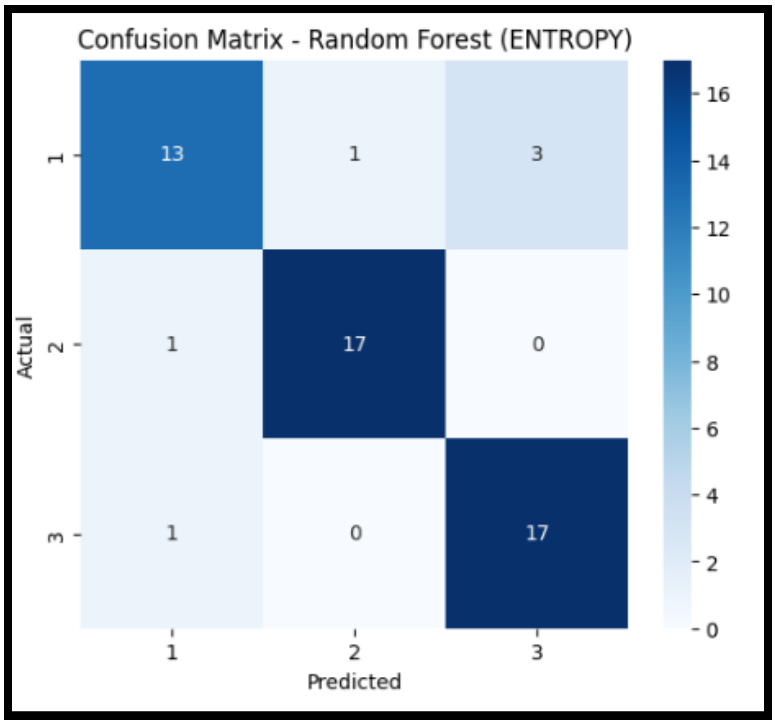
Data Training Complete

INPUT: Model Evaluation

```
accuracy_entropy = accuracy_score(y_test, y_pred_entropy)

print("Accuracy (Entropy):", accuracy_entropy)

cm_entropy = confusion_matrix(y_test, y_pred_entropy)

plt.figure(figsize=(6, 5))

sns.heatmap(

    cm_entropy,

    annot=True,

    fmt="d",

    cmap="Blues",

    xticklabels=np.unique(y),

    yticklabels=np.unique(y)

)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix - Random Forest (ENTROPY)")

plt.show()
```

OUTPUT:

Accuracy (Entropy): 0.8867924528301887



HYPERPERAMETER(ENTROPY):

INPUT: n_estimator

```
# n_estimators values

n_values = [50, 100,200,300,500]

accuracies = []

# Train & evaluate

for n in n_values:

    model = RandomForestClassifier(

        n_estimators=n,

        criterion="entropy",

        random_state=42

    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)

    accuracies.append(acc)

    print(f"ENTROPY | n_estimators={n} | Accuracy={acc:.4f}")
```

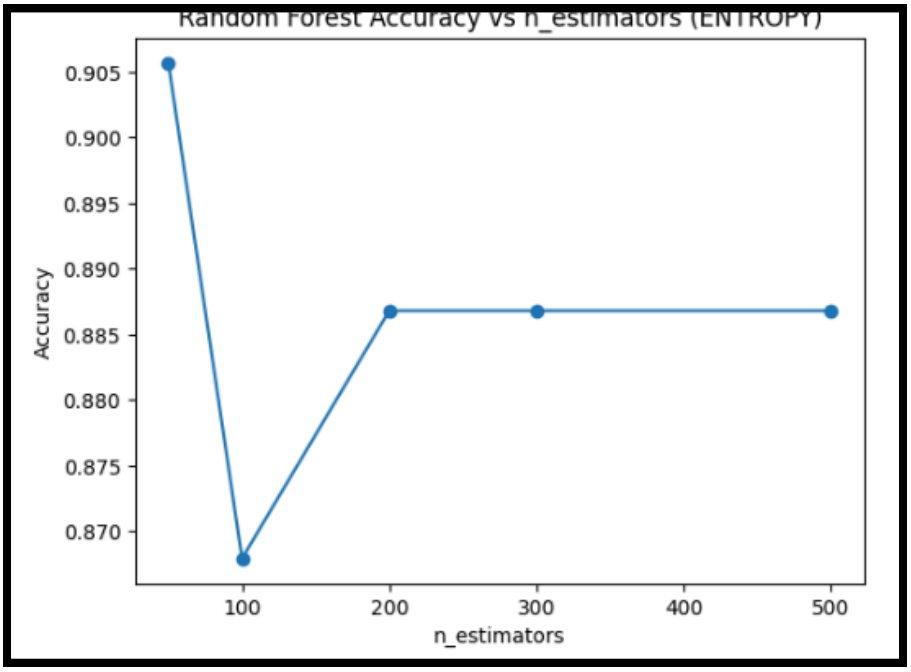
```
# Plot graph

plt.figure()

plt.plot(n_values, accuracies, marker='o')

plt.xlabel("n_estimators")

plt.ylabel("Accuracy")

plt.title("Random Forest Accuracy vs n_estimators (ENTROPY)")

plt.show()
```

OUTPUT:

ENTROPY | n_estimators=50 | Accuracy=0.9057

ENTROPY | n_estimators=100 | Accuracy=0.8679

ENTROPY | n_estimators=200 | Accuracy=0.8868

ENTROPY | n_estimators=300 | Accuracy=0.8868

ENTROPY | n_estimators=500 | Accuracy=0.8868